LAB 3 – Architectural Justification

NAME – PURAV N KHINCHA
SRN – PES1UG24CS349
SECTION – 5F

We chose **Microservices Architecture** for the Internal Microservice Catalog & Health Portal System.

Reason 1: Independent Scaling of Heterogeneous Workloads

The system's components have fundamentally different load profiles. The Health Prober must run a lightweight probe against every registered service every 30 seconds — a high-frequency, I/O-bound workload that grows linearly with the number of services monitored (NFR-001 targets smooth performance up to 200 services). By contrast, the Service Catalog handles infrequent, low-volume CRUD operations (registering or editing a microservice). A monolithic or layered design would force both workloads to scale together; microservices let us scale the Health Prober horizontally to handle catalog growth without over-provisioning the rest of the system.

Reason 2: Fault Isolation for a Monitoring System

A system whose entire purpose is detecting outages cannot itself become an additional point of failure. If the Alerting Service crashes or is mid-deployment, engineers must still be able to browse the dependency graph and search API documentation — those capabilities must not degrade because an unrelated component failed. Microservices architecture isolates failures to individual services; a Layered or Client-Server design would risk a single backend failure taking down the entire portal, which is unacceptable for a tool teams rely on during incident response.

Security Advantage

Each microservice exposes only a narrow, well-defined API surface and can enforce its own authentication and authorization boundary independently. For example, the Health Prober only needs read access to endpoint URLs from the Service Catalog — it has no need for, and is never granted, access to user-management or alerting configuration data. This narrow-interface design limits the blast radius of a compromised service and supports the role-based access control called for in NFR-002, since permissions can be scoped per service rather than granted broadly across a single shared codebase.

Performance Benefit

Because the Health Prober is deployed and scaled independently, it can be replicated across multiple instances to probe hundreds of services in parallel every 30 seconds without adding latency to the Portal UI or the Availability Engine's calculations. In a Layered architecture, this same probing workload would compete for resources within a single shared business-logic layer, risking slower graph rendering and search response times under peak load — directly undermining the < 2s render target in NFR-001.